Robust Steganography from Large Language Models

Neil Perry, Sanket Gupte, Nishant Pitta, and Lior Rotem

Stanford University
{naperry,sanketg,lrotem}@cs.stanford.edu

arXiv:2504.08977v1 [cs.CR] 11 Apr 2025

Abstract. Recent steganographic schemes, starting with Meteor (CCS'21), rely on leveraging large language models (LLMs) to resolve a historically-challenging task of disguising covert communication as "innocent-looking" natural-language communication. However, existing methods are vulnerable to "re-randomization attacks," where slight changes to the communicated text, that might go unnoticed, completely destroy any hidden message. This is also a vulnerability in more traditional encryption-based stegosystems, where adversaries can modify the randomness of an encryption scheme to destroy the hidden message while preserving an acceptable covertext to ordinary users. In this work, we study the problem of **robust steganography**. We introduce formal definitions of weak and strong robust LLM-based steganography, corresponding to two threat models in which natural language serves as a covertext channel resistant to realistic re-randomization attacks. We then propose two constructions satisfying these notions. We design and implement our steganographic schemes that embed arbitrary secret messages into natural language text generated by LLMs, ensuring recoverability even under adversarial paraphrasing and rewording attacks. To support further research and real-world deployment, we release our implementation and datasets for public use.

1 Introduction

In authoritarian regimes, even the mere act of encryption can be a death sentence. In such settings, *steganography*—the art of embedding hidden messages within innocent-looking communications [28,19]—provides a critical lifeline. While cryptography secures content against unauthorized readers, steganography goes further by concealing the very existence of the communication. In repressive environments, this distinction is critical: even the act of encrypting a message can raise suspicion or trigger punitive measures.

To ground this discussion, imagine a world where a dictatorship mandates that all communication be conducted using state-issued public-key encryption, and the regime retains a copy of every citizen's secret key. In such a setting, encryption offers no privacy—messages are effectively plaintext to the state. Yet remarkably, Horel et al. [16] showed that even under such extreme surveillance, hidden communication is still possible. By leveraging the randomness inherent in encryption, one can encode secret bits via rejection sampling, choosing ciphertexts whose structure encodes information while remaining indistinguishable from ordinary ciphertexts.

However, such techniques are fragile. A simple countermeasure available to the regime is ciphertext re-randomization: decrypt the message and re-encrypt the same plaintext with fresh randomness before forwarding it. This effectively erases the hidden message, without altering the visible content of the communication. Similar fragility appears in modern steganographic schemes based on large language models (LLMs), where information is hidden in sampled tokens conditioned on a secret seed [21]. These methods can also be disrupted by semantically neutral edits—such as rewording, paraphrasing, or reformatting—that preserve the apparent meaning of the message but destroy the encoding.

This brittleness motivates a new desideratum for steganographic systems: *robustness*. Intuitively, a robust stegosystem should withstand realistic modifications to the covertext, preserving

the hidden message so long as the overt meaning remains unchanged. This models the behavior of a sophisticated adversary—e.g., a censorship regime—that seeks to strip hidden content while maintaining the flow of normal communication.

In this paper, we initiate the study of robust steganography. We formalize adversarial models in which covertexts are subject to modification—ranging from localized perturbations to full semantic paraphrasing—and define both weak and strong notions of robustness corresponding to these capabilities.

We then present two new steganographic constructions that aim to meet these definitions. The first perturbs the sampling distribution of an LLM to encode hidden bits in a key-dependent but undetectable manner, and is weakly robust to local edits. The second uses semantic embeddings and locality-sensitive hashing to embed information in the semantic space of the message, enabling strong robustness to paraphrasing and meaning-preserving edits.

Both constructions are implemented and evaluated. We measure their resistance to realistic tampering, assess the overhead required to achieve robustness, and provide practical guidance for deployment. Our watermarking-based system survives most classes of attacks—even when up to 20% of the covertext is modified—while maintaining a recovery rate of approximately 90% or higher. The embedding-based scheme resists attacks far more effectively than the watermarking-based schemes– surviving up to 50% covertext modifications while requiring 100x fewer tokens for the covertext per bit hidden. Additionally, the embedding system on average costs < 1¢ per hidden byte of data. Our code and experiments are available at https://github.com/NeilAPerry/robust_steganography.

2 Background and Related Work

Steganography—the practice of hiding secret information within other innocent-looking messages—has long been studied as a means of secure communication in adversarial environments. In this paper, we consider a particularly extreme case: a world in which a dictatorship assigns each citizen a public-secret key pair, mandates the use of these keys for all communication, and retains a copy of every citizen's secret key. In such a setting, the dictator can read all communications as if they were transmitted in plaintext.

Horel et al. [16] recently demonstrated that even under such draconian conditions, private, undetectable communication remains possible. Their construction operates by selecting the encryption randomness in a way that depends on the secret message. More concretely, suppose Alice wishes to send a secret bit $b \in \{0, 1\}$ to Bob, with whom she shares a secret key k unknown to the dictator. She repeatedly encrypts an innocent-looking message m with fresh randomness until she obtains a ciphertext ct such that $H(k, ct) = b$, where H is a suitable hash function. This method is effective provided it meets two key criteria:

- The ciphertext should remain indistinguishable from a normal encryption of m to the dictatorship, even though it possesses Bob's secret key (but not k); and
- For any given m and b, a ciphertext ct satisfying $H(k, ct) = b$ should be found after only a small number of trials.

Horel et al. showed that both conditions can be met under standard cryptographic assumptions—specifically, that the encryption scheme is semantically secure to any party lacking the

recipient's secret key. They further demonstrated that similar ideas can be used to undetectably execute key exchange protocols, even in this hostile setting.

The general idea of using rejection sampling over a high-entropy stochastic channel for steganography dates back to early foundational work [9], and variations of this idea have appeared in nearly all subsequent rigorous stegosystems [15,37,2].

However, such schemes are inherently fragile. A straightforward countermeasure available to the dictatorship is ciphertext re-randomization: the regime decrypts the ciphertext ct, re-encrypts the same plaintext m with fresh randomness, and forwards the new ciphertext $\hat{ct}$ to Bob. This re-randomization destroys any steganographically embedded content, while preserving the plaintext, and therefore does not interfere with honest communication. In light of this, we say that the scheme of Horel et al. lacks *robustness*, since "reasonable modifications" to the covertext—in this case, the ciphertext—destroy the hidden message. We formally define robustness and reasonable modifications in Section 4.2.

The situation worsens if the regime bans randomized encryption altogether, mandating the use of deterministic encryption [4,31,8], or even prohibits encryption entirely. In such settings, any steganographic embedding must occur directly in the plaintext channel. This possibility has prompted extensive research into steganographic schemes that embed hidden messages within natural language text [38,11,6,35,33,42,47,45], or other natural data such as images [41,39,12,3,46,26].

The idea of hiding messages within common language is not new—humans have been doing so for thousands of years, with historical accounts dating back to Ancient Greece [20]. However, nearly all such systems, including the modern ones cited above, are heuristic in nature and lack rigorous security guarantees. Their fragility arises from two central challenges recognized in the formal study of steganography:

- A well-defined probability distribution over the vocabulary of the covertext language is required, conditioned on prior history; and
- Each block of text must contain sufficient entropy to allow secure hiding of information.

Natural languages such as English pose problems for both requirements. The distribution over valid continuations of a sentence is hard to formalize, highly context-dependent, and typically exhibits low entropy in predictable settings. For instance, the sentence "At the museum I saw a Tyrannosaurus" is overwhelmingly likely to be completed with "Rex," offering little room for information embedding due to the determinism of the next token.

Meteor [21] addresses these challenges by using modern large language models (LLMs) to both produce and analyze text. In Meteor, both sender and receiver rely on the same LLM to model token distributions and determine where information can be safely embedded. Specifically, the sender uses a ciphertext as the seed to condition the LLM and begins generating text from that state. At each token step, the model's predicted distribution is used to assess whether sufficient entropy is present to embed bits. If so, the sender samples from a subset of the distribution in a way that encodes the secret message. The receiver, using the same LLM, can detect high-entropy positions and recover the embedded bits.

Meteor's approach offers several advantages. It ensures that steganographic messages are only embedded when statistically safe, that the covertext closely resembles human writing, and that both sender and receiver can independently determine where embedding is feasible. Nonetheless, Meteor inherits the fundamental vulnerability of all previous systems discussed thus far: it is not robust. Minor changes to the generated text—such as replacing words with synonyms, adding or